Particle Swarm Optimization

Feature Selection for DDOS attack detection in IOT:

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

X, y = make_classification(
    n_samples=2000,
    n_features=20,
    n_informative=12,
    n_redundant=4,
    n_classes=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_fit, X_val, y_fit, y_val = train_test_split(
    X_train, y_train, test_size=0.25, random_state=42, stratify=y_train
)

scaler = StandardScaler()
X_fit = scaler.fit_transform(X_fit)
X_val = scaler.transform(X_val)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)

n_particles = 10
n_features = X.shape[1]

positions = np.random.randint(0, 2, (n_particles, n_features))
```

```python
    positions = np.random.randint(0, 2, (n_particles, n_features))
    velocities = np.random.rand(n_particles, n_features)

    pbest = positions.copy()
    pbest_scores = np.zeros(n_particles)

    def fitness(position):
        selected = np.where(position == 1)[0]

        if len(selected) == 0:
            return 0

        model = MLPClassifier(
            hidden_layer_sizes=(64, 32),
            max_iter=500,
            random_state=42,
            early_stopping=True
        )

        model.fit(X_fit[:, selected], y_fit)
        predictions = model.predict(X_val[:, selected])

        return f1_score(y_val, predictions)

    for i in range(n_particles):
        pbest_scores[i] = fitness(positions[i])

    gbest = pbest[np.argmax(pbest_scores)].copy()
    gbest_score = max(pbest_scores)

    for _ in range(10):
        for i in range(n_particles):
            r1 = np.random.rand(n_features)
            r2 = np.random.rand(n_features)
```

```
            velocities[i] = (
                0.7 * velocities[i]
                + 1.5 * r1 * (pbest[i] - positions[i])
                + 1.5 * r2 * (gbest - positions[i])
            )

            probabilities = 1 / (1 + np.exp(-velocities[i]))
            positions[i] = (np.random.rand(n_features) < probabilities).astype(int)

            score = fitness(positions[i])

            if score > pbest_scores[i]:
                pbest[i] = positions[i].copy()
                pbest_scores[i] = score

            if score > gbest_score:
                gbest = positions[i].copy()
                gbest_score = score

    selected = np.where(gbest == 1)[0]

    model = MLPClassifier(
        hidden_layer_sizes=(128, 64, 32),
        max_iter=1000,
        random_state=42,
        early_stopping=True
    )

    model.fit(X_train[:, selected], y_train)
    predictions = model.predict(X_test[:, selected])

    print("Selected Features:", len(selected))
    print("Accuracy:", accuracy_score(y_test, predictions))
    print("Precision:", precision_score(y_test, predictions))
    print("Recall:", recall_score(y_test, predictions))
```

1/1

Output:

```
Selected Features: 12
Accuracy: 0.9175
Precision: 0.9322916666666666
Recall: 0.8994974874371859
F1 Score: 0.9156010230179028
```